2026

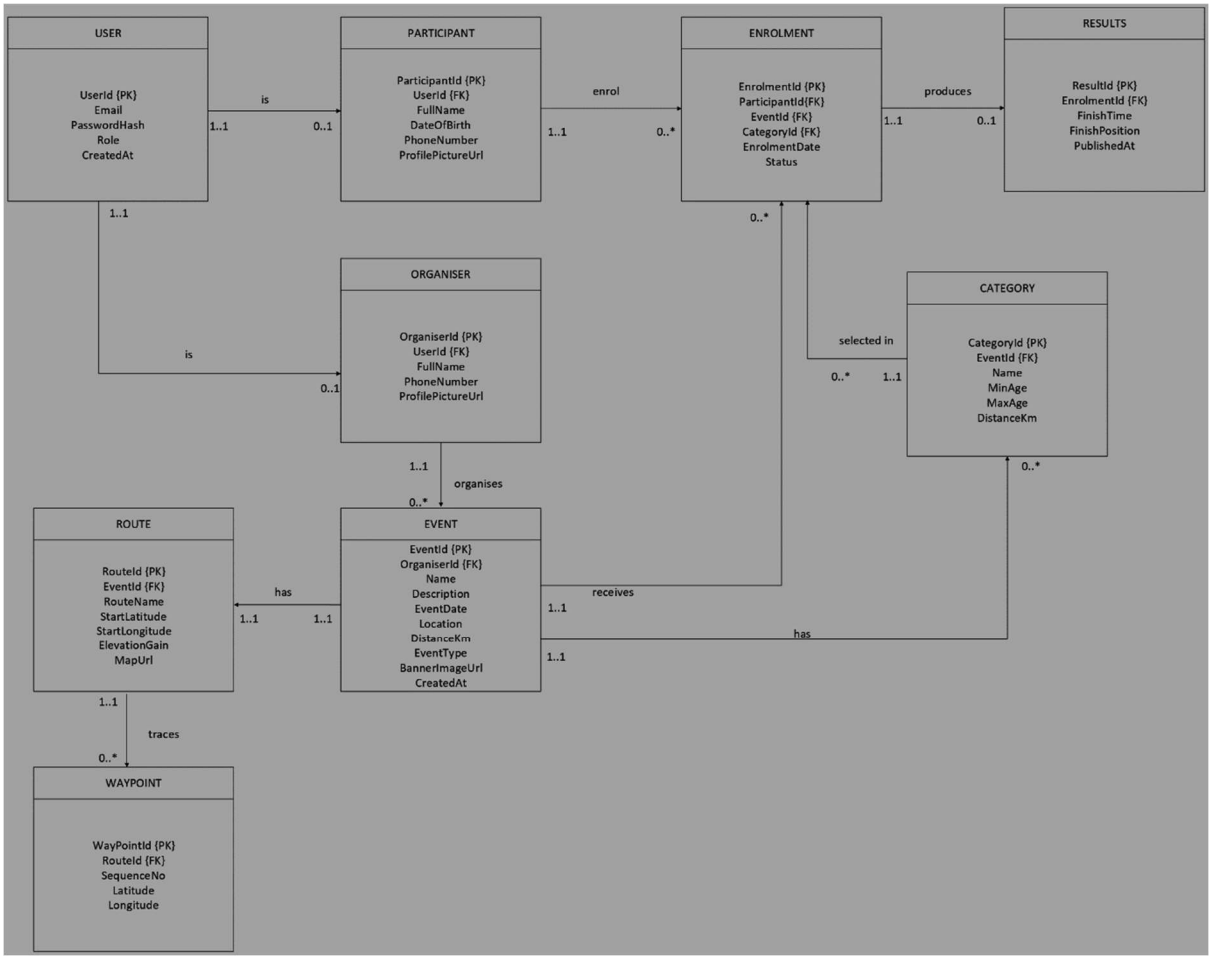
NAMES: Dakalo Mudau

STUDENT NUMBER: 10467108

MODULE: Programming 2B

MODULE CODE: PROG6212

Section A – Entity Relationship Diagram (ERD)



The User table serves as the central authentication hub, storing only login credentials (Email, PasswordHash) and role information. This design choice ensures that authentication logic remains separate from business logic, improving security and maintainability. The Role field with a check constraint restricts values to either 'Organiser' or 'Participant', enabling role-based access control throughout the system. The CreatedAt timestamp provides audit capability for tracking when accounts were created.

The Organiser and Participant tables extend the User table through one-to-one relationships, storing role-specific profile information. This separation eliminates the need for nullable fields, as seen in the original design where DateOfBirth had to be NULL for organisers. The Participant table includes mandatory DateOfBirth for age validation during event enrolment, while the Organiser table contains only relevant fields like FullName, PhoneNumber, and ProfilePictureUrl. Both tables use UserId as a foreign key with a unique constraint, ensuring each user can have only one role in the system.

The Event table forms the core of the system, storing all event-related information created by organisers. Each event links to an organiser through OrganiserId, enabling proper ownership tracking for CRUD operations. The table includes comprehensive fields such as Name, Description, EventDate, Location, DistanceKm, and EventType with a check constraint

restricting values to 'Run', 'Walk', or 'Cycle'. The BannerImageUrl field supports Azure Blob Storage integration for event banners, as specified in Part 3 requirements.

The Category table enables organisers to define age or distance-based categories for each event. Each event can have multiple categories, allowing participants to choose appropriate divisions based on their age or preferred distance. The MinAge and MaxAge fields support automated age validation, while DistanceKm allows for distance-based categorisation in events like marathons where participants might choose between full or half marathons.

The Route table stores official route information for events, with a unique constraint ensuring each event has only one route. The StartLatitude and StartLongitude fields store GPS coordinates for map rendering in the MVC front-end, while ElevationGain captures uphill data for route difficulty assessment.

The WayPoint table complements the Route table by storing ordered GPS points that trace the route's actual path. The SequenceNo field maintains point order, and the composite unique constraint on RouteId and SequenceNo prevents two points from sharing the same position in the sequence, ensuring accurate route rendering.

The Enrolment table resolves the many-to-many relationship between participants and events. Each enrolment records a participant registering for an event under a specific category. The composite unique constraint on ParticipantId and EventId prevents duplicate enrolments, while the Status field with check constraints ('Pending', 'Confirmed', 'Cancelled') enables enrolment lifecycle management. The EnrolmentDate timestamp tracks when the enrolment was created.

The Results table captures finish times and positions for participants who complete events. The unique constraint on EnrolmentId ensures each enrolment produces only one result, maintaining data integrity. The FinishTime field stores completion times using the time data type, while FinishPosition tracks ranking. The PublishedAt timestamp enables organisers to control when results become visible to participants and the public.

Section B – API Endpoint Plan

HTTP method	Route	Description	Role Required	Request body	Expected response
POST	/api/auth/register	It creates a new account for the participant.	None (public)	{fullName, email, password, dateOfBirth, role, phoneNumber}	201 Created – new participant id returned | 400 Bad Request – missing/invalid fields | 409 Conflict – account already exists
POST	/api/auth/login	It authenticates a user and starts a session with user id and role.	None (public)	{email, password}	200 OK – session has started, user id and role returned| 401 Unauthorized – incorrect email or password
POST	/api/auth/logout	It ends the current session.	Any (logged in)	None	200 OK – session ended; no data returned
GET	/api/users/me	It returns the logged in user's profile.	Any	None	200 OK – logged-in user's profile returned| 401 Unauthorized – no active session
PUT	/api/users/me	It updates the logged in user's profile.	Any	{fullName, phoneNumber, profilePictureUrl}	200 OK – updated profile returned | 400 Bad Request – invalid field values

GET	/api/events	It lists all the upcoming events with optional filters.	None (public)	None	200 OK – a list of upcoming events returned
GET	/api/events/{id}	It returns full details for one event.	None (public)	None	200 OK – full event detail returned | 404 Not Found – no returned event with this id
POST	/api/events	It creates a new event.	Organiser	{name, description, eventDate, location, distanceKm, eventType}	201 Created – new event id returned| 400 Bad Request – missing/invalid fields | 403 Forbidden – logged-in user is not an organiser
PUT	/api/events/{id}	It updates an event the organiser owns.	Organiser	{name, description, eventDate, location, distanceKm, eventType}	200 OK – updated event returned | 403 Forbidden _ organiser does not own this event | 404 Not Found – event does not exist
DELETE	/api/events/{id}	It deletes an event the organiser owns.	Organiser	None	204 No Content – event deleted, nothing returned | 403 Forbidden – organiser does not own the event | 404 Not Found – event does not exist

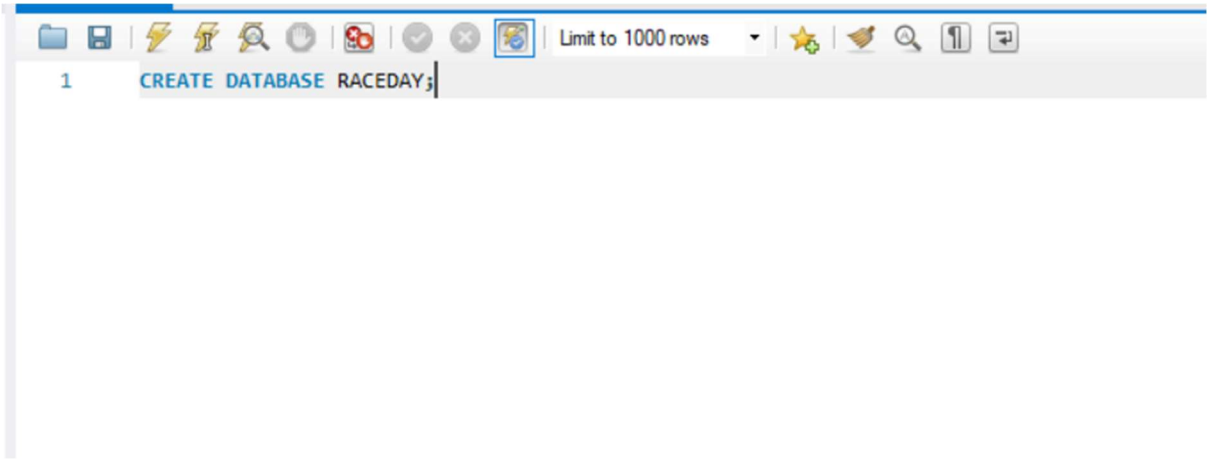
POST	/api/events/{id}/banner	It uploads a banner image to azure blob storage sent as a multipart/ form data file.	Organiser	{file}	200 OK – banner image URL returned| 400 Bad Request – invalid/missing file| 403 Forbidden – organiser does not own this event
GET	/api/events/{id}/categories	It lists all the categories for an event.	None (public)	None	200 OK – list of categories for the event returned
POST	/api/events/{id}/categories	It adds a category to the event.	Organiser	{name, minAge, maxAge, distanceKm}	201 Created – new category id returned | 403 Forbidden – organiser does not own this event| 404 Not Found – event does not exist
PUT	/api/categories/{id}	It updates an existing category.	Organiser	{name, minAge, maxAge, distanceKm}	200 OK – updated category returned | 403 Forbidden – organiser does not own the event | 404 Not Found – category does not exist
DELETE	/api/categories/{id}	It removes a category from an event.	Organiser	None	204 No Content – category deleted| 403 Forbidden - organiser does not own the event| 404 Not Found – category does not exist

POST	/api/events/{id}/enrolments	It enrols the logged in participant under a chosen category.	Participant	{categoryId}	201 Created – new enrolment id and status returned| 400 Bad Request – invalid category | 404 Not Found – event or category does not exist | 409 Conflict – already enrolled in this event
GET	/api/users/me/enrolments	It lists the participant's enrolment and status.	Participant	None	200 OK – list of the participant's own enrolments and their status returned
GET	/api/events/{id}/enrolments	It lists all the participants enrolled in an event the organiser owns.	Organiser	None	200 OK – list of participants enrolled in the event returned| 403 Forbidden – organiser does not own this event| 404 Not Found – event does not exist
DELETE	/api/enrolments/{id}	It cancels the participant's enrolment.	Participant	None	204 No Content – enrolment cancelled| 403 Forbidden – enrolment does not belong to this participant | 404 Not Found

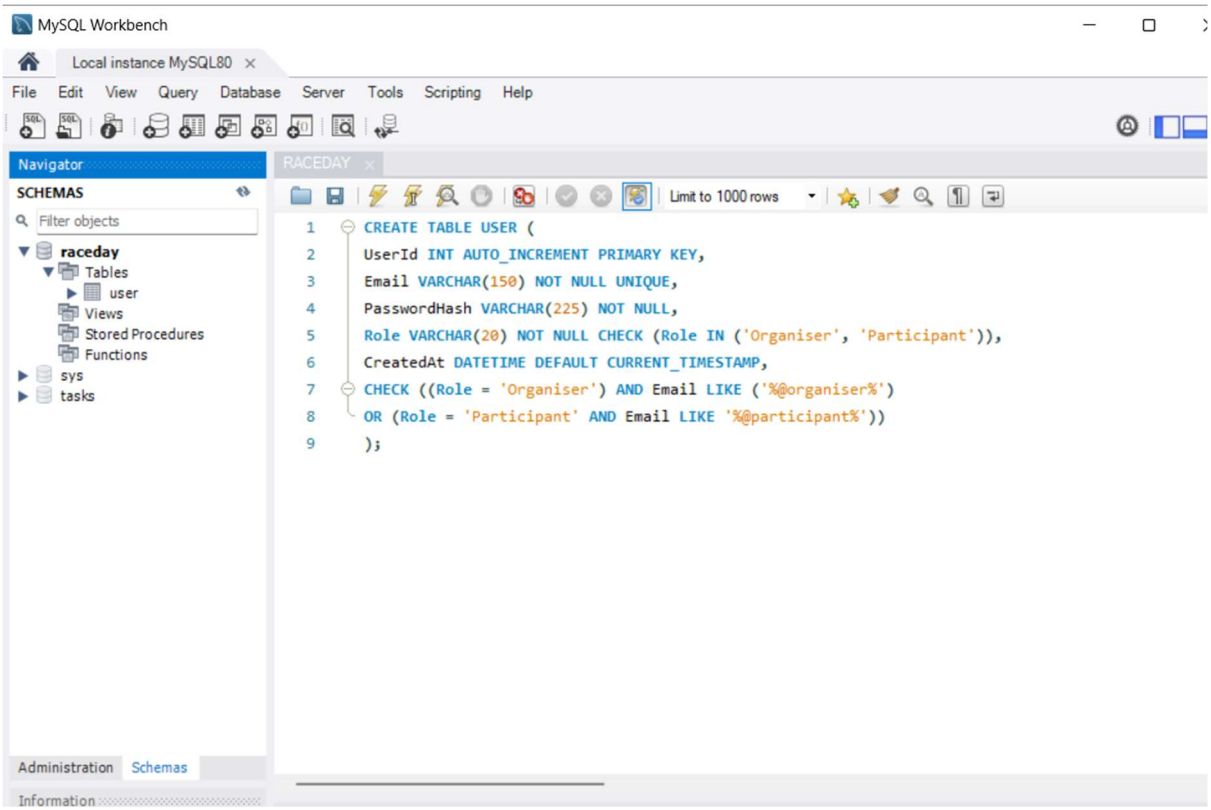
					– enrolment does not exist
POST	/api/enrolments/{id}/results	It captures a finish time and position after the event.	Organiser	{finishTime, finishPosition}	201 Created – new result returned | 400 Bad Request – invalid time or position value | 403 Forbidden – organiser does not own the event | 404 Not Found – enrolment does not exist
GET	/api/users/me/results	It returns the participant's race history.	Participant	None	200 OK – list of the participant's result returned
GET	/api/events/{id}/results	It returns the full published results for an event.	None (public)	None	200 OK – list of published results for the event returned | 404 Not Found – event does not exist
GET	/api/events/{id}/route	It returns route details and waypoints for map rendering.	None (public)	None	200 OK – route details and ordered waypoints returned| 404 Not Found – no route set for this event
POST	/api/events/{id}/route	It adds or updates route details and waypoints.	Organiser	{routeName, startLatitute, startLongitude, elevationGainM, mapUrl, waypoints []}	200 OK – saved route and waypoints returned| 400 Bad Request – invalid coordinates | 403 Forbidden – organiser does not own the event| 404

					Not Found – event does not exist
GET	/api/events/{id}/weather	It fetches a live forecast from the OpenWeather API using stored coordinates.	None (public)	None	200 OK – forecast returned| 404 Not Found – event or stored coordinates missing | 502 Bad Gateway – OpenWeather service unavailable or API key invalid

SECTION C – SQL Database Script

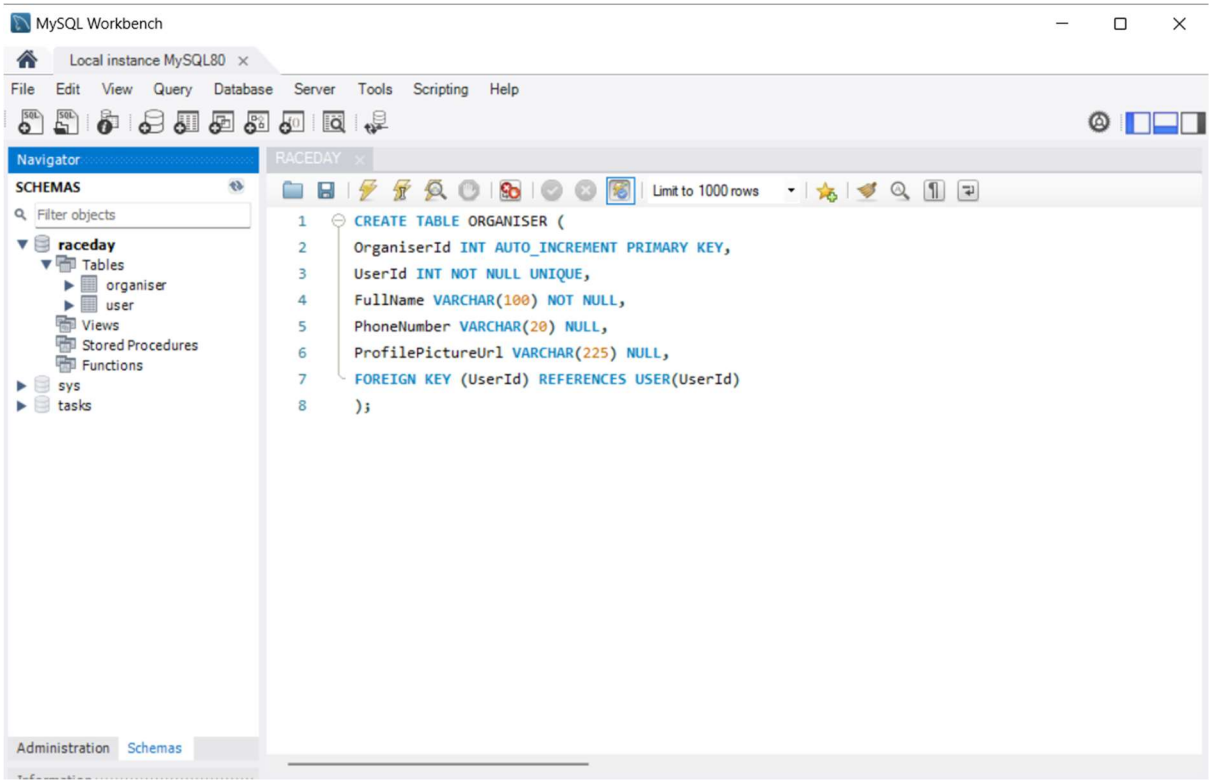


The statement creates database named RACEDAY which will contain 9 tables for the race day system.

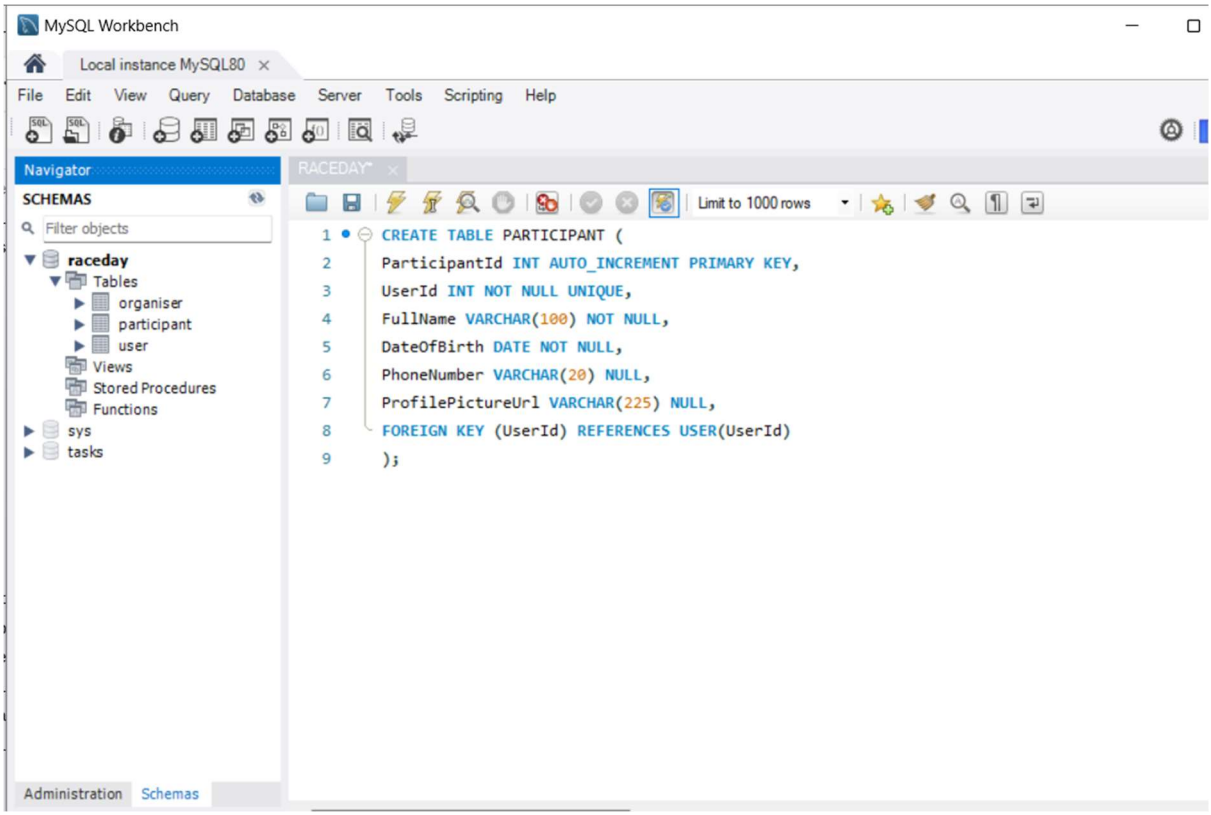


It acts as a gateway for both roles login and authentication. It contains only what each account needs to login, in this case it needs the role, passwordhash and email, ensuring that all columns are unique to prevent duplicate accounts. As the two roles require different values, e.g., the date of birth is only needed for participants meaning that their profile information is stored in the organiser or participant tables. The check constraint ensures that the emails contain “@organiser” for the organiser and “@participant” for the partcipant to easily differentiate the two roles. The

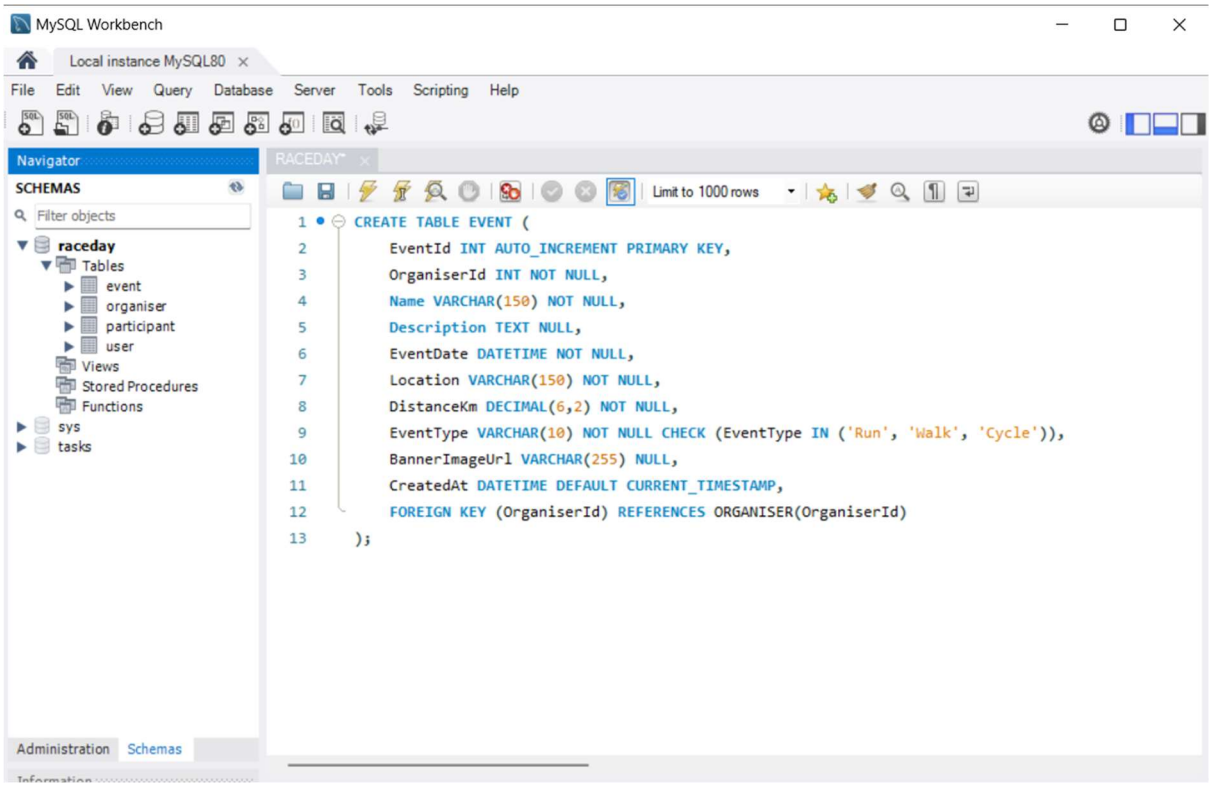
registrations logic will validate this first and return a specific error message if ever the user enters a wrong email or if it does not contain the above.



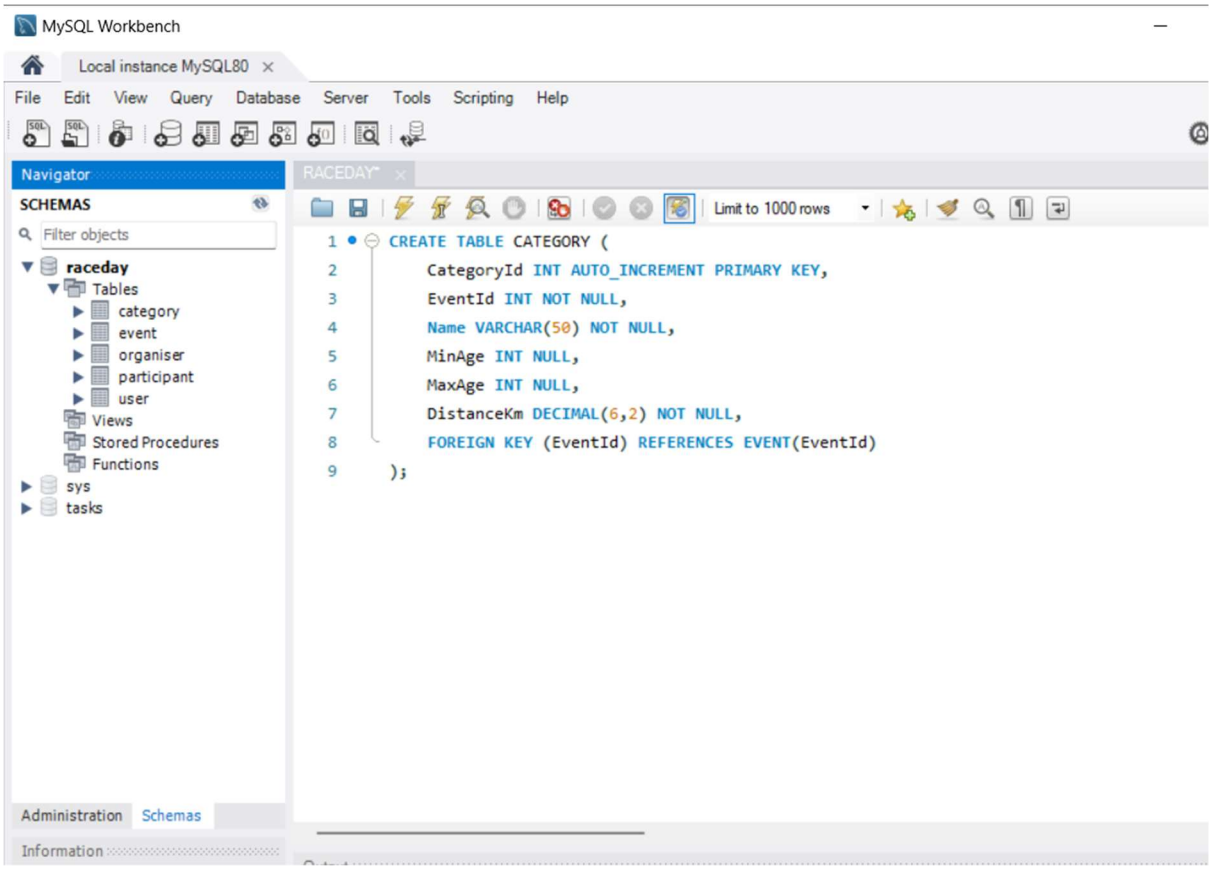
Stores profile details specific to Organiser accounts. OrganiserId is a plain auto-incrementing integer the 'O' prefix (e.g., 'O1') is applied when displaying it in Part 3's MVC front end (e.g., $"O{OrganiserId}" in C#), not stored in the database itself. Keeping the stored key, a plain integer avoids extra database complexity (no triggers or hidden columns needed) while still letting the interface show a readable, role-distinguishable code. UserId links back to the matching login row in USER and is UNIQUE.



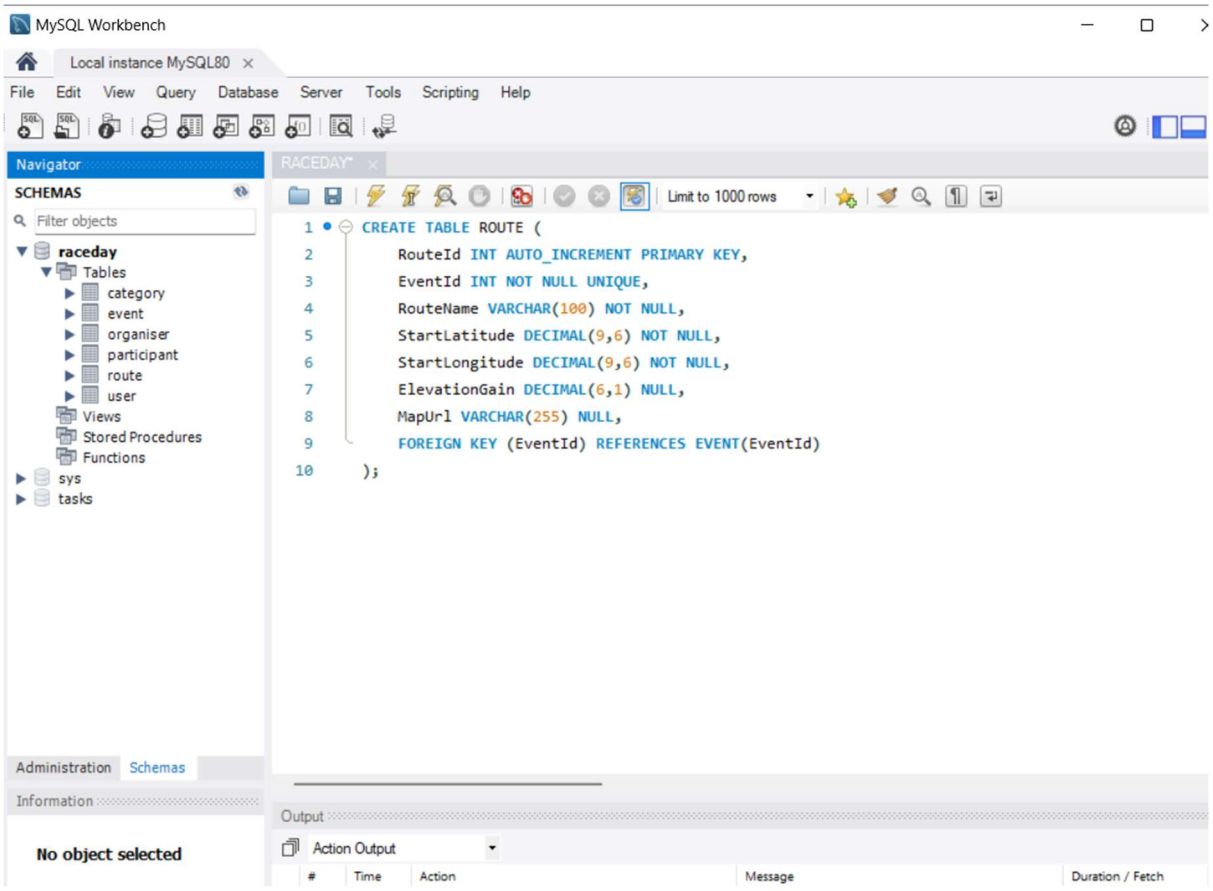
Stores profile details specific to Participant accounts. ParticipantId is a plain auto-incrementing integer, same reasoning as OrganiserId the 'P' prefix (e.g., 'P1') is applied in Part 3's display logic, not stored here. UserId links back to the matching login row in USER and is UNIQUE. DateOfBirth is NOT NULL, since only Participants ever have a row in this table at all this is what removes the need for a nullable DateOfBirth column that a single combined USER table would have required. The DateOfBirth is used to check if the user is eligible to enter a certain event based on the age constriction set by the organiser.



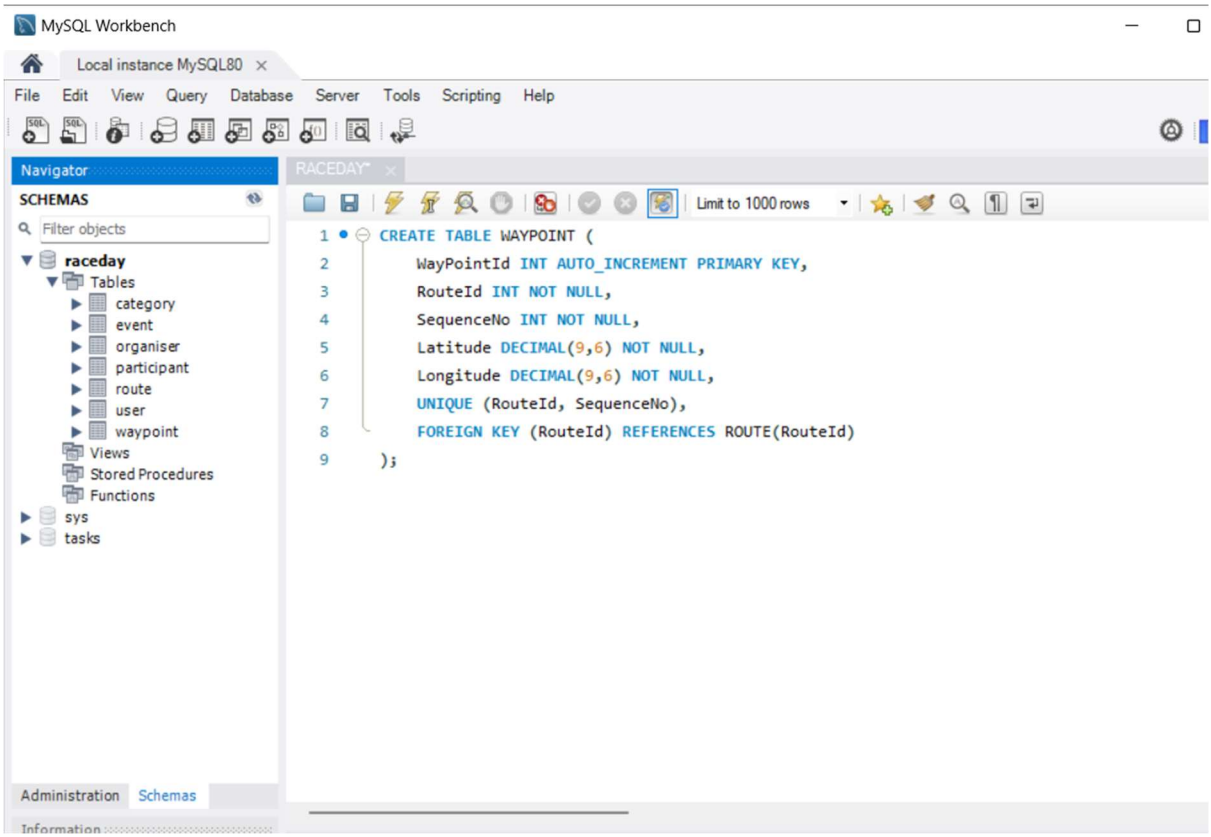
The event table will be used to store events created by organisers with an auto incremented id, which includes the organiserId as a foreign key to link the two tables. The check constraint is used to check if the event type is run, walk, or cycle and the createdAt attribute will automatically set the current time when a record has been added. The table will also allow the organisers to set a brief description detail about the event which includes the event date, location and distanceKm and the banner image URL to support the Azure Blob Storage integration in part 3, which will then be used to upload the event banners.



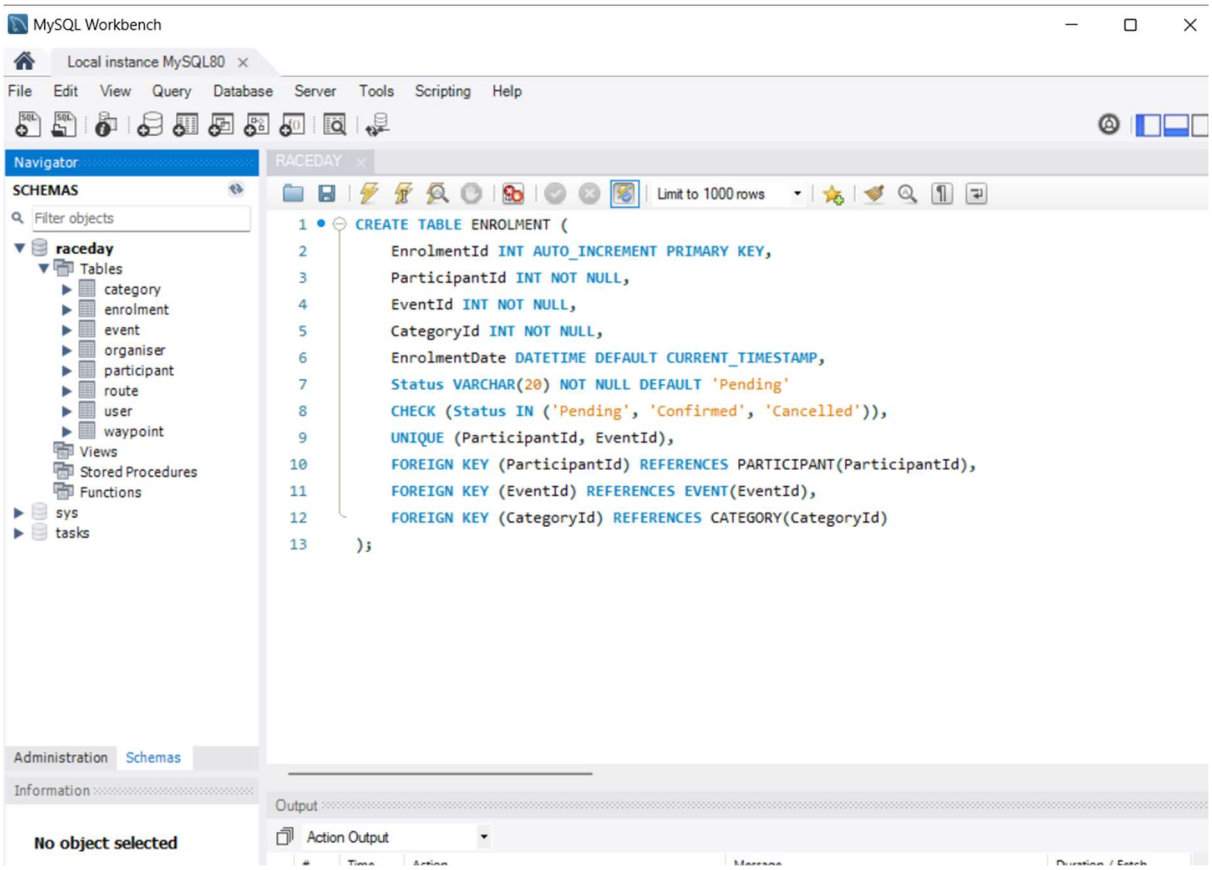
The category table stores the different categories that an organiser creates for each event, such as age-based groups like Junior or Senior, or distance-based options. Each category is linked to a specific event using the EventId foreign key, and the table includes MinAge and MaxAge columns to define age restrictions, while DistanceKm stores the category distance. The CategoryId serves as the primary key, and the foreign key constraint ensures that every category belongs to a valid event, allowing one event to have many categories.



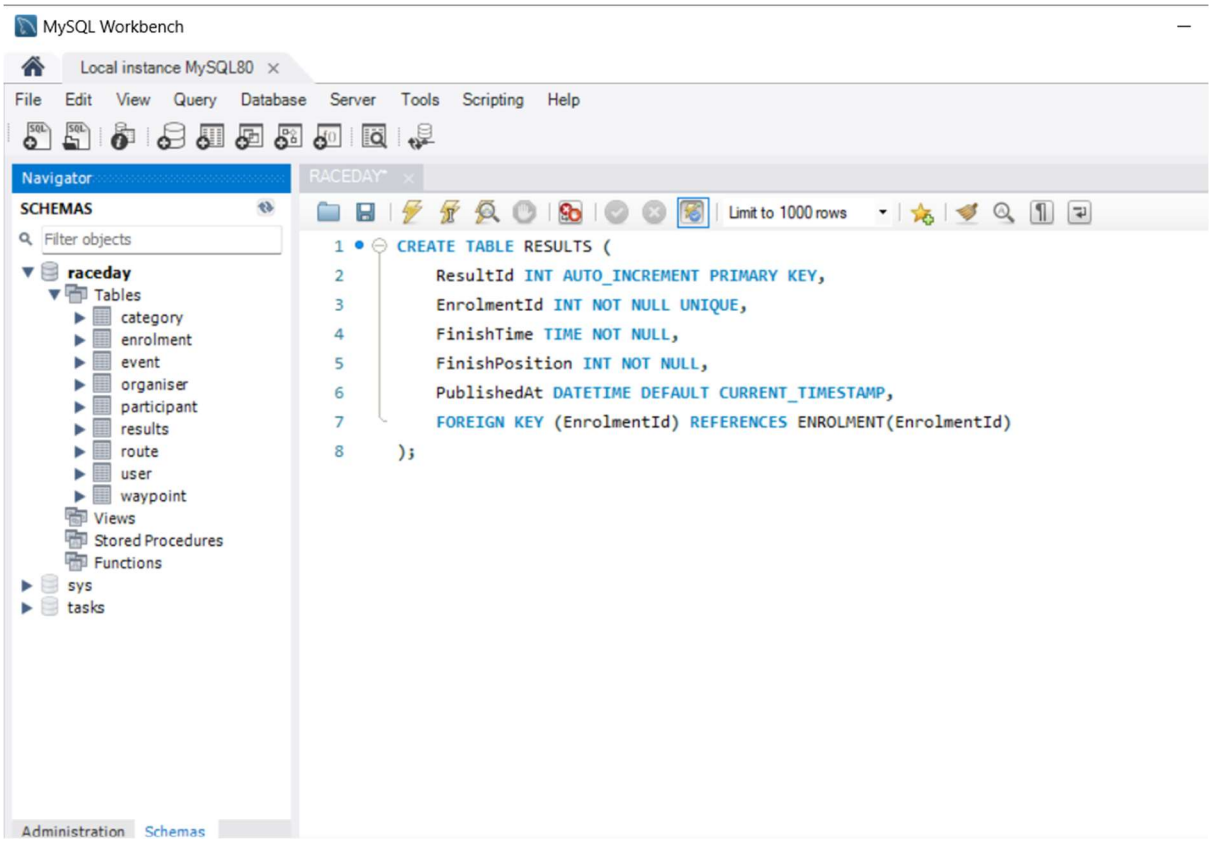
The route table stores the official route information for each event. Each event can have only one route, enforced by the unique constraint on EventId. The table includes RouteId as the primary key, RouteName to identify the route, and StartLatitude and StartLongitude columns to store GPS coordinates for map rendering in the MVC front-end. The ElevationGain column stores the total uphill elevation in meters to help participants understand the route difficulty, while MapUrl provides an optional link to an external mapping service. The foreign key ensures each route belongs to a valid event.



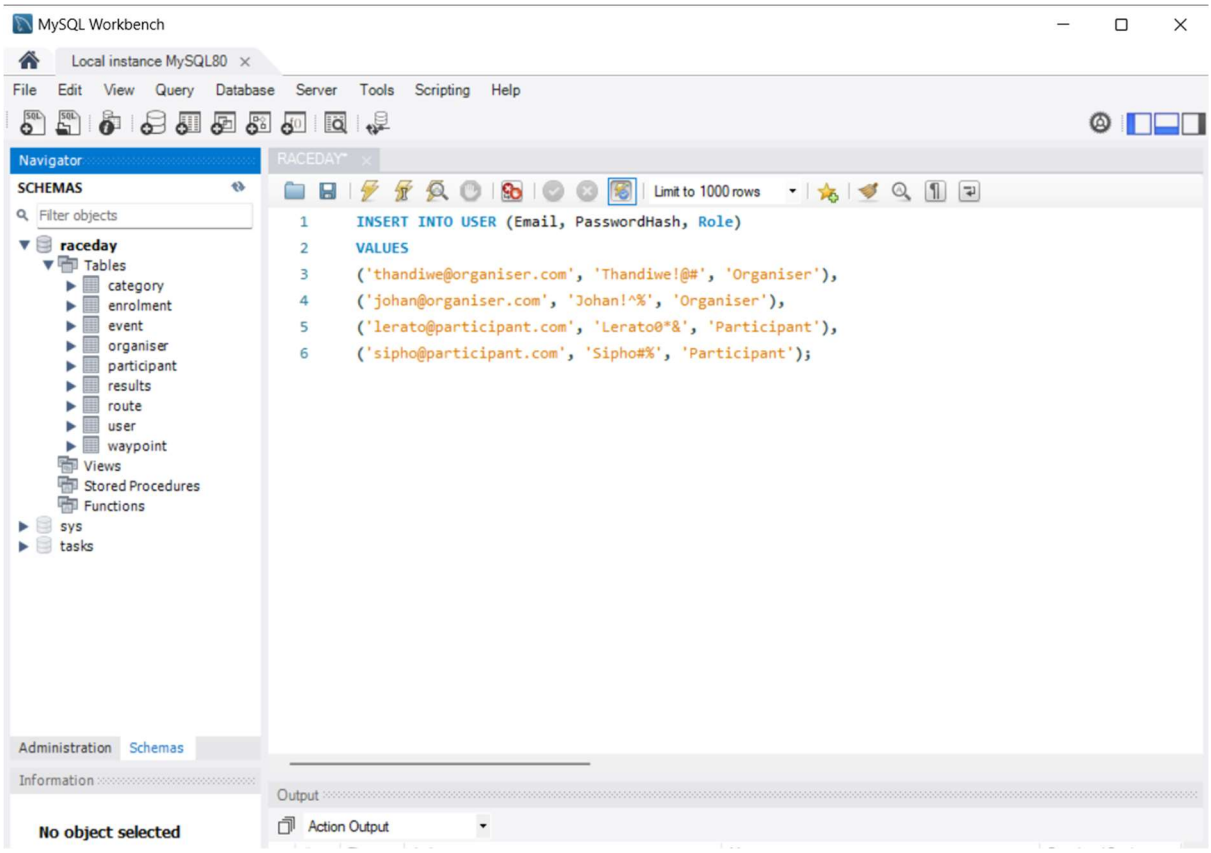
The waypoint table stores individual GPS points that trace a route on a map. Each waypoint belongs to a specific route using the RouteId foreign key, and the SequenceNo column determines the order in which the points should be connected to draw the route correctly. The Latitude and Longitude columns store the exact coordinates for each point. The unique constraint on RouteId and SequenceNo ensures that each route has a unique order of waypoints, preventing duplicate sequence numbers. The foreign key ensures that every waypoint is linked to a valid route, allowing one route to have many waypoints.



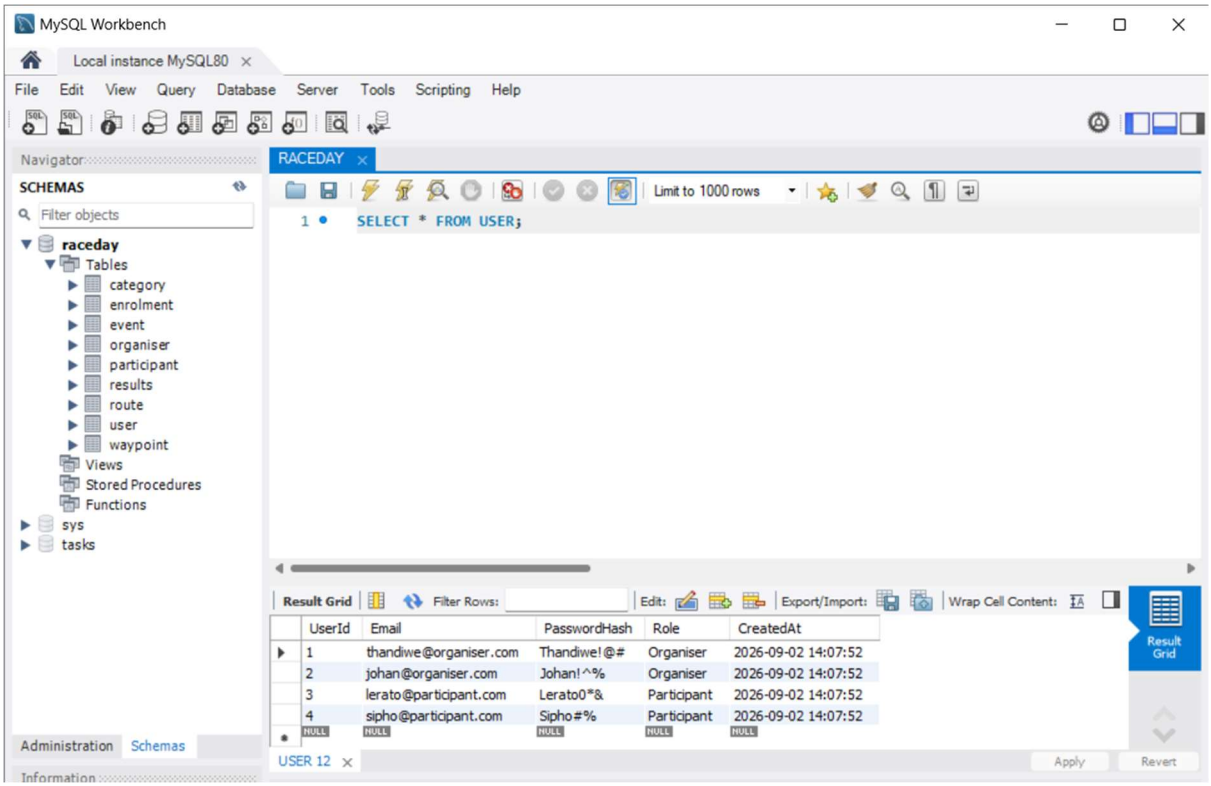
The enrolment table records when a participant registers for an event and selects a category. Each enrolment links to the participant through ParticipantId, the event through EventId, and the chosen category through CategoryId. The EnrolmentDate automatically sets the current timestamp when the record is created. The Status column defaults to 'Pending' and only allows 'Pending', 'Confirmed', or 'Cancelled' through a check constraint. The unique constraint on ParticipantId and EventId prevents a participant from enrolling in the same event twice. The foreign keys ensure that every enrolment links to valid participants, events, and categories, allowing each participant to have many enrolments.



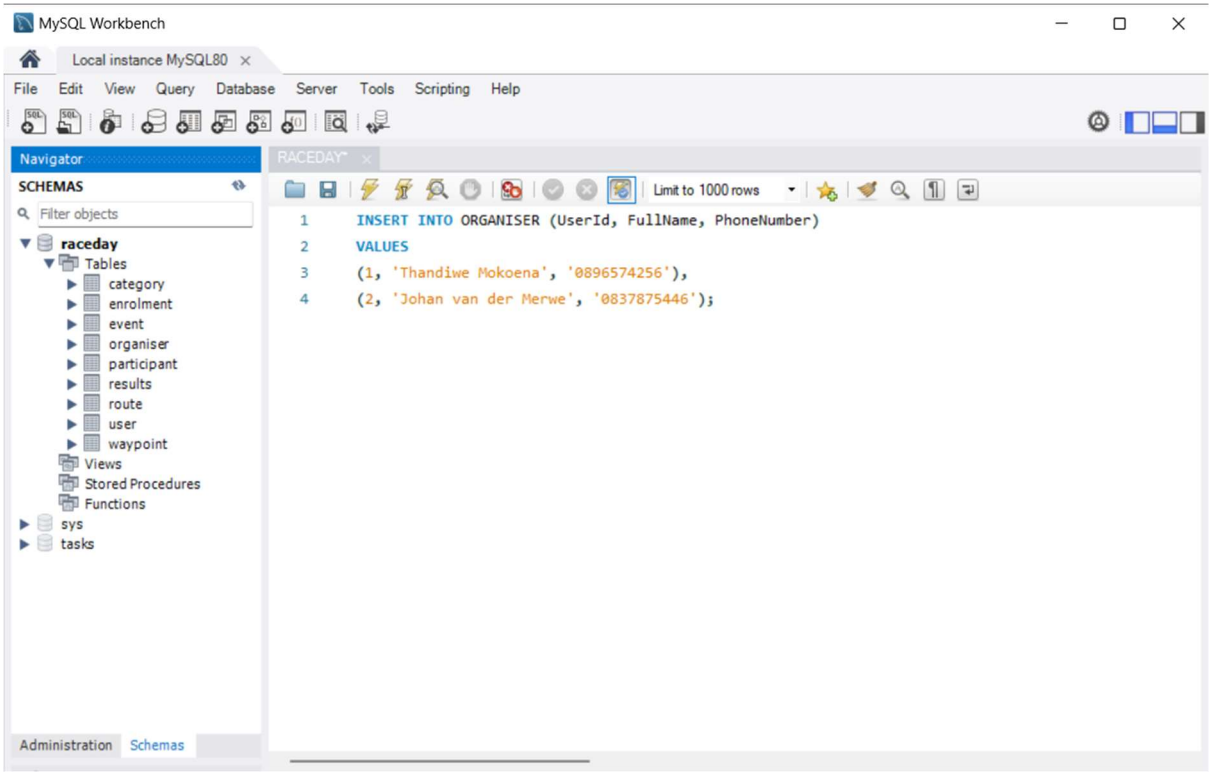
The results table stores finish times and positions for participants who have completed an event. Each result is linked to a specific enrolment through the EnrolmentId foreign key, and the unique constraint ensures that one enrolment can have only one result. The FinishTime column records the participant's completion time, while FinishPosition stores their ranking in the event. The PublishedAt column automatically sets the current timestamp when the result is recorded. The foreign key ensures that every result is linked to a valid enrolment, allowing participants to view their personal race history across completed events.



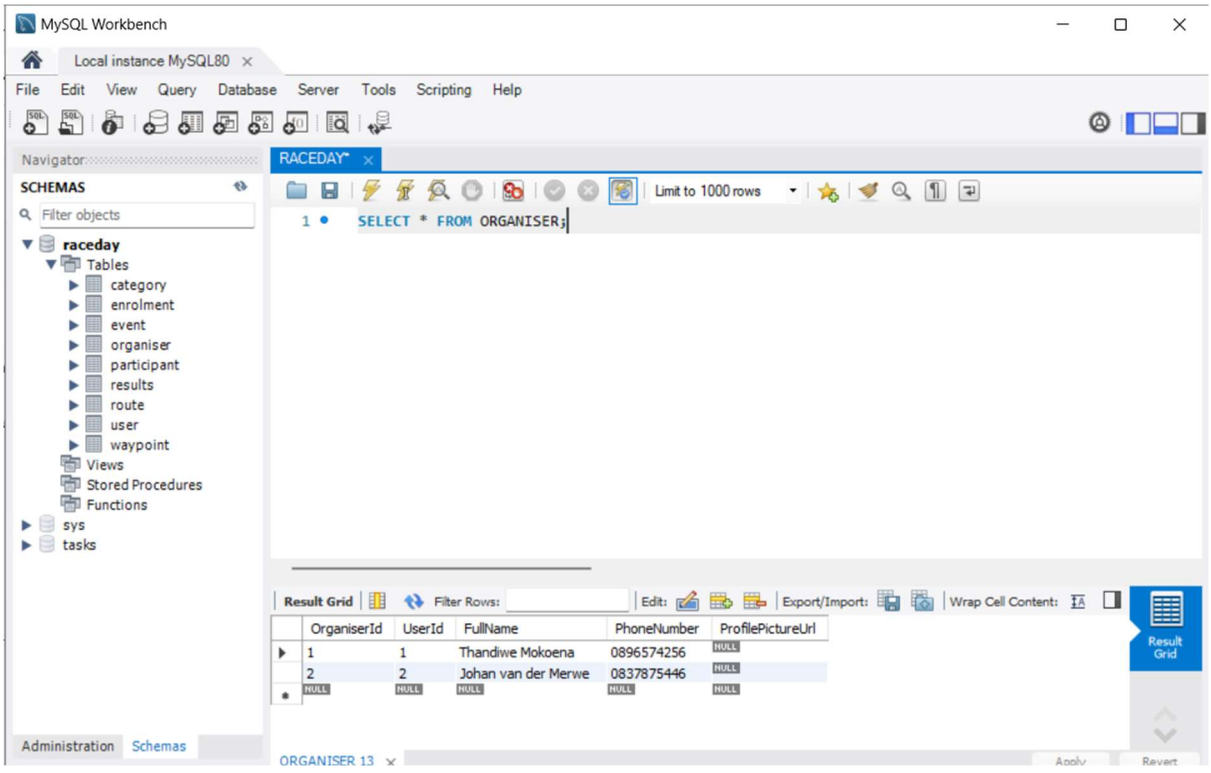
The insert statement shows four login accounts with only login fields and the email follow a role-based domain pattern that will be validated in the registration logic.



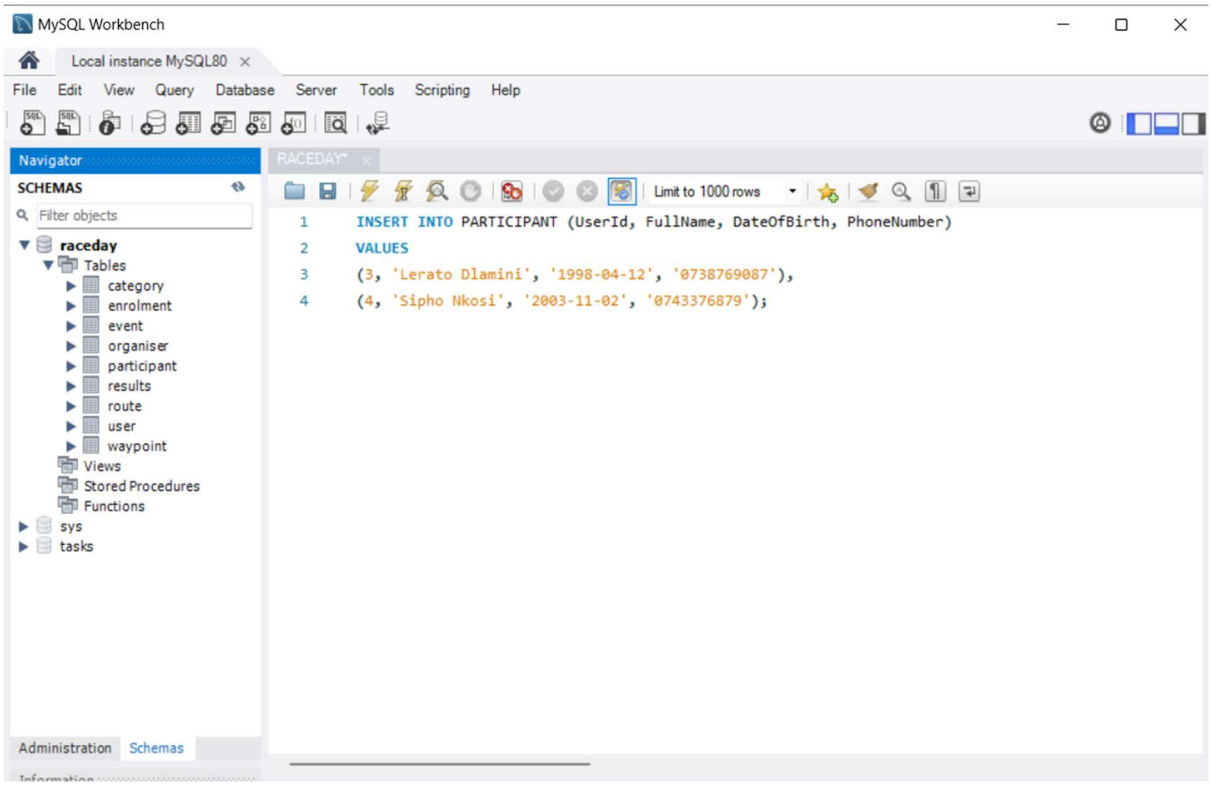
The select statement displays the four login accounts stored.



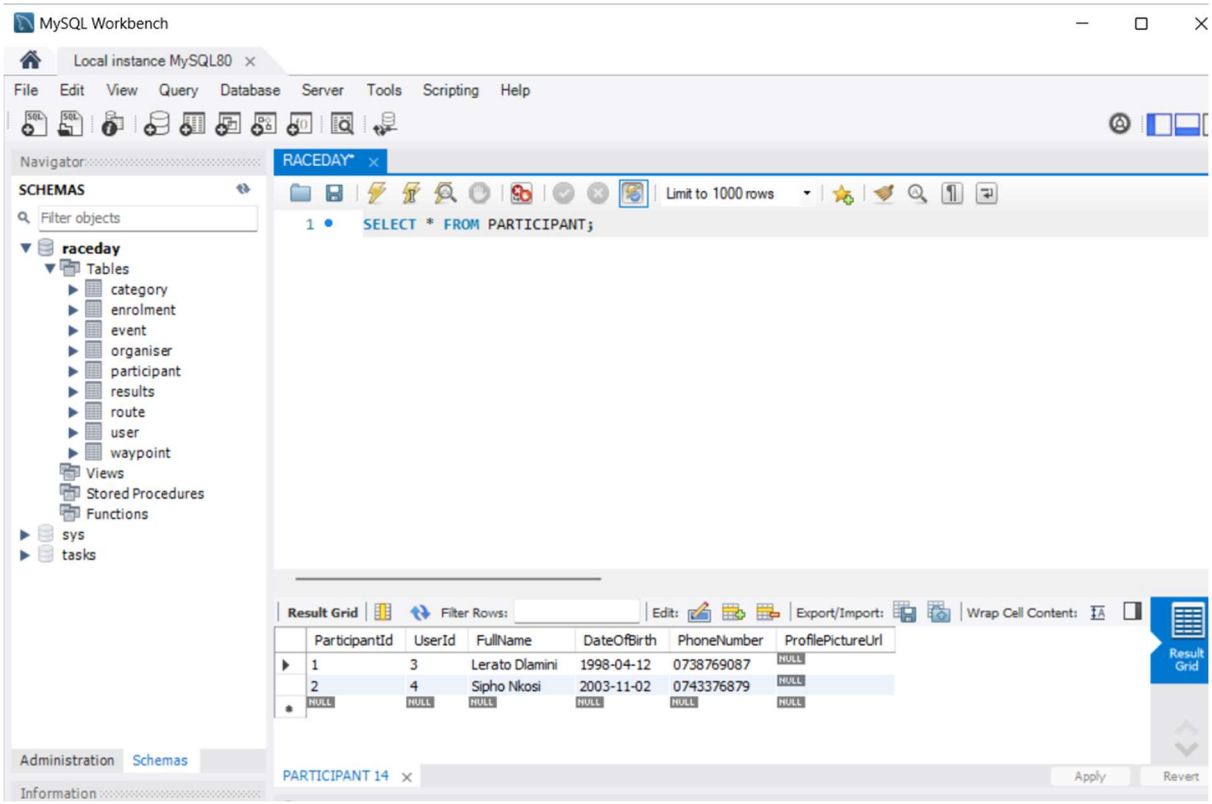
The insert statement shows the two organisers stored linking them back to their login fields in the user table.



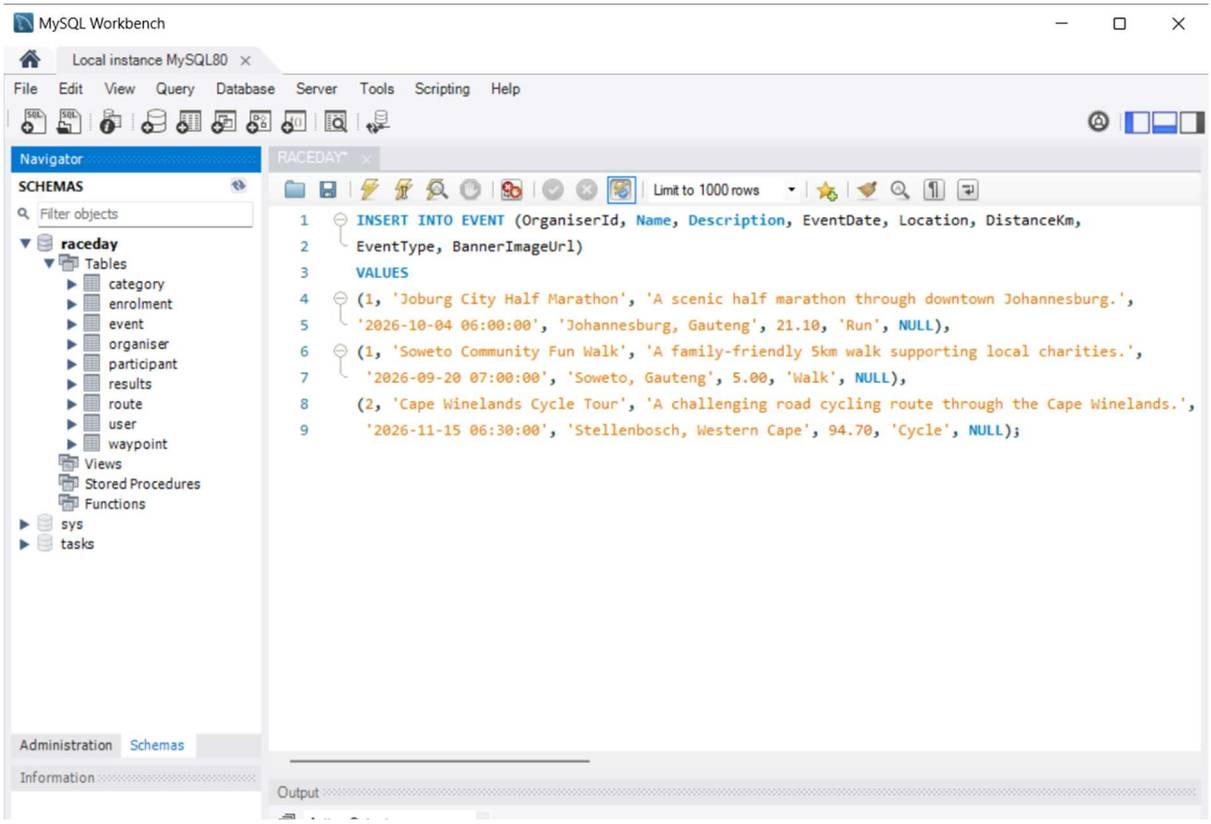
The select statement displays the organisers stored.



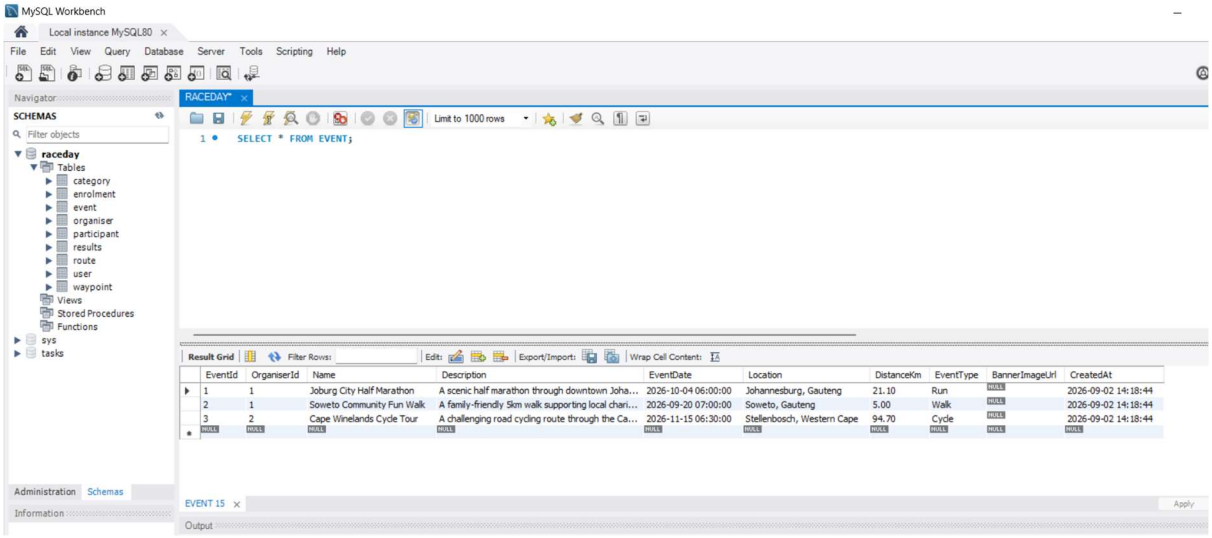
The insert statement adds the profile rows for the two participant accounts including date of births for the age-restriction category to be tested.



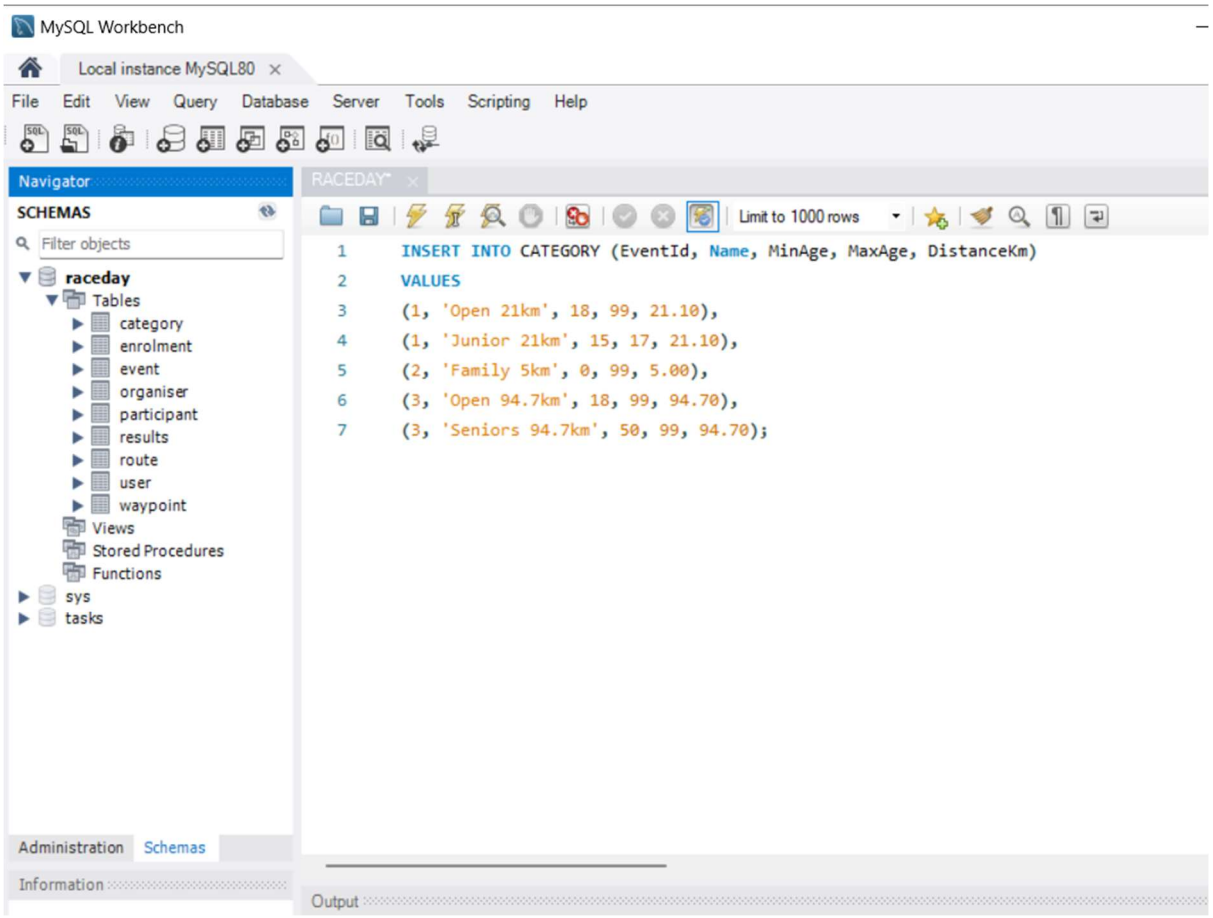
The select statement displays the participants stored.



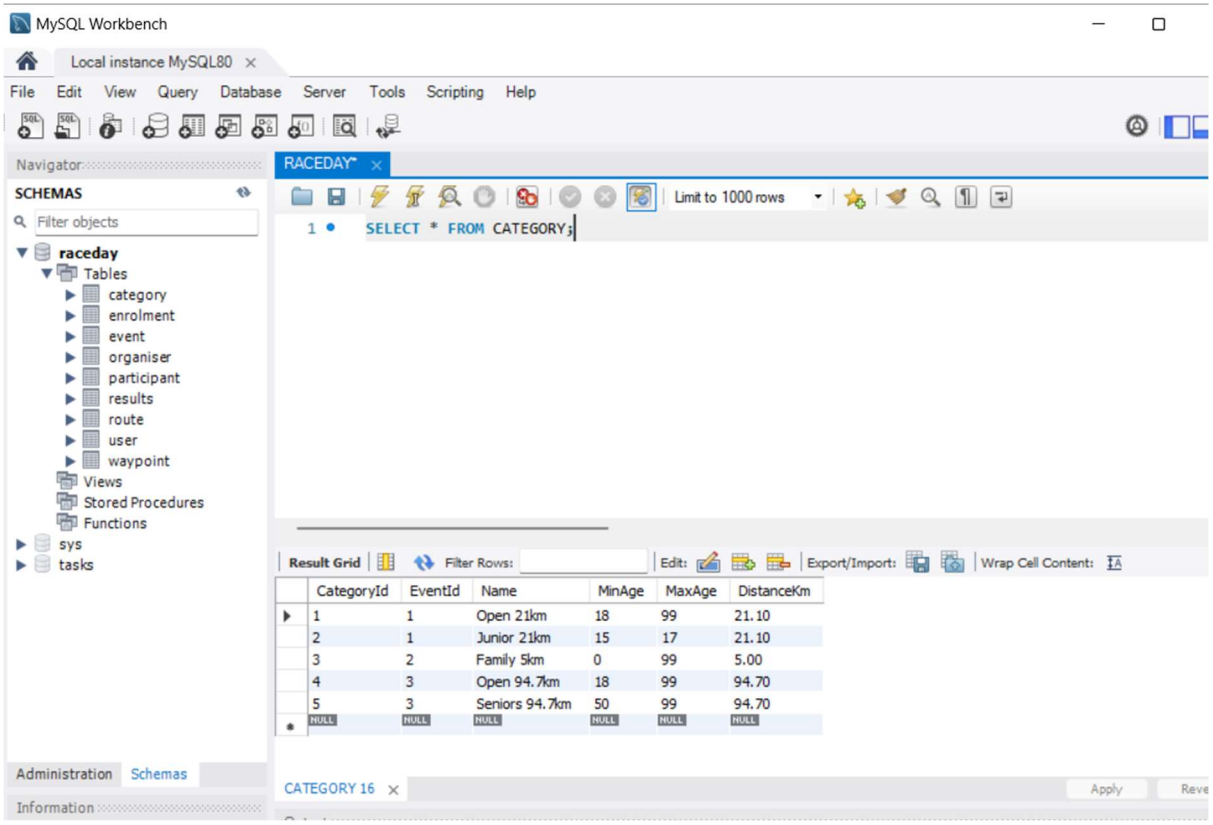
The insert statement adds 3 events each owned by organisers through the organiserId as a foreign key in the table.



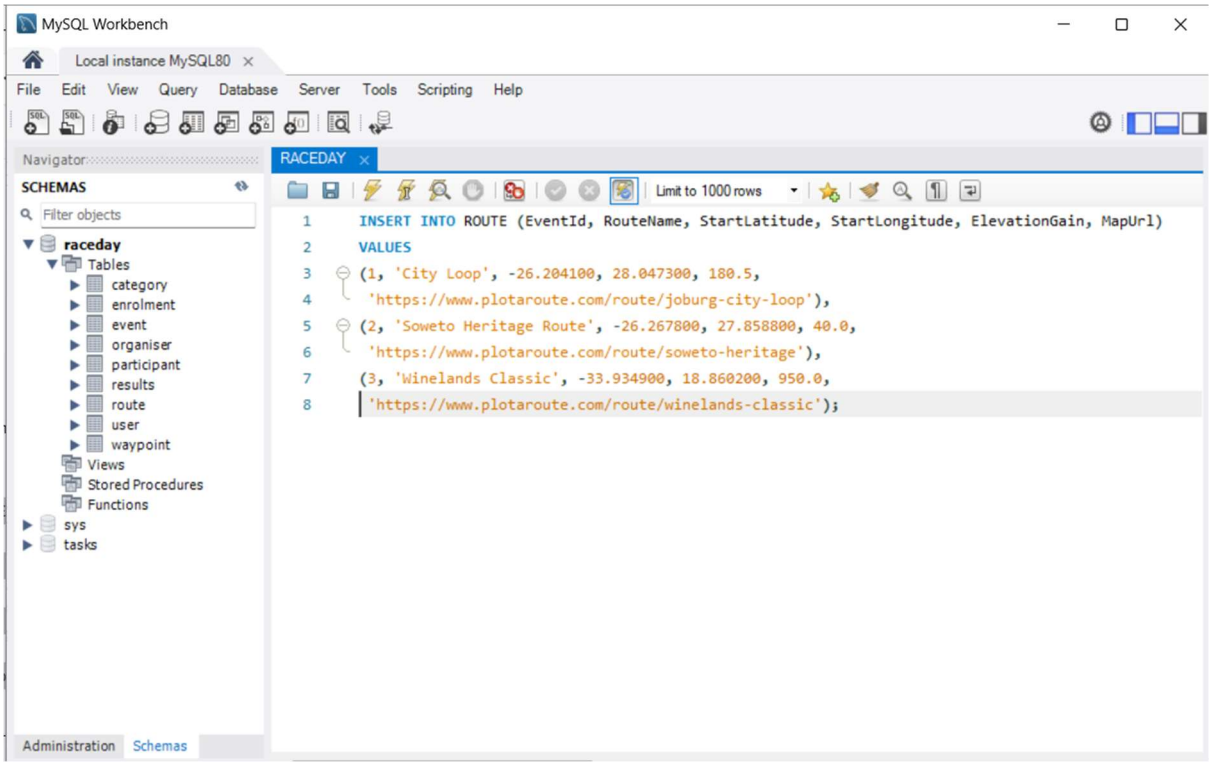
The select statement displays the events owned by organisers.



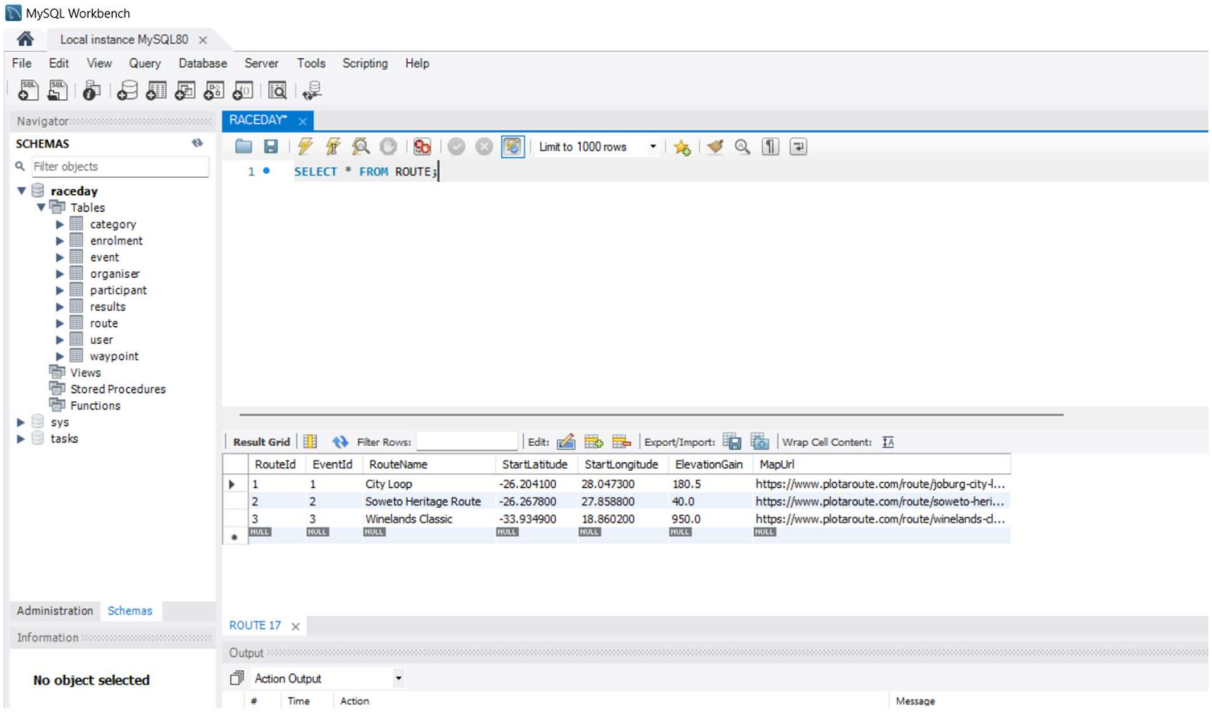
The insert statement adds age-based categories including the age-restricted junior 21km to demonstrate the existents of the date of birth attribute in the participant table.



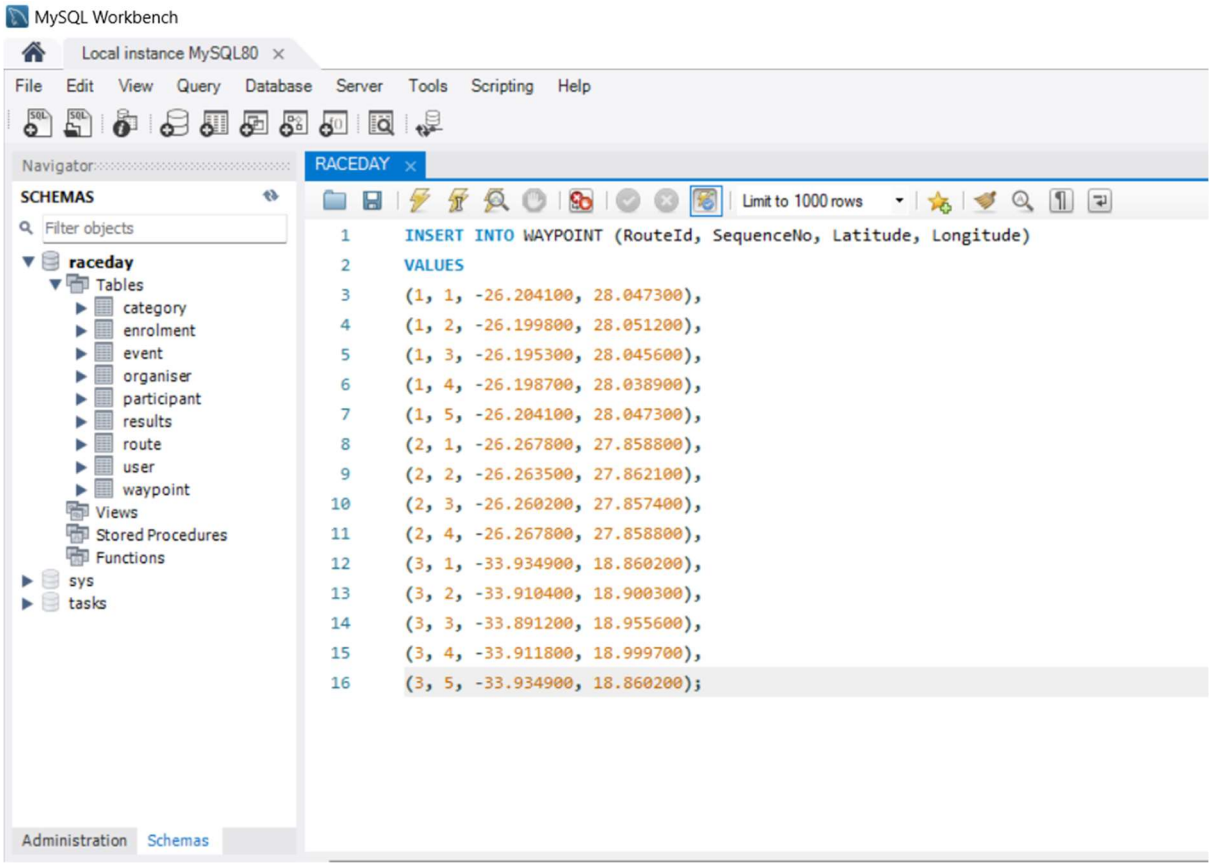
The select statement displays the age-based categories per event.



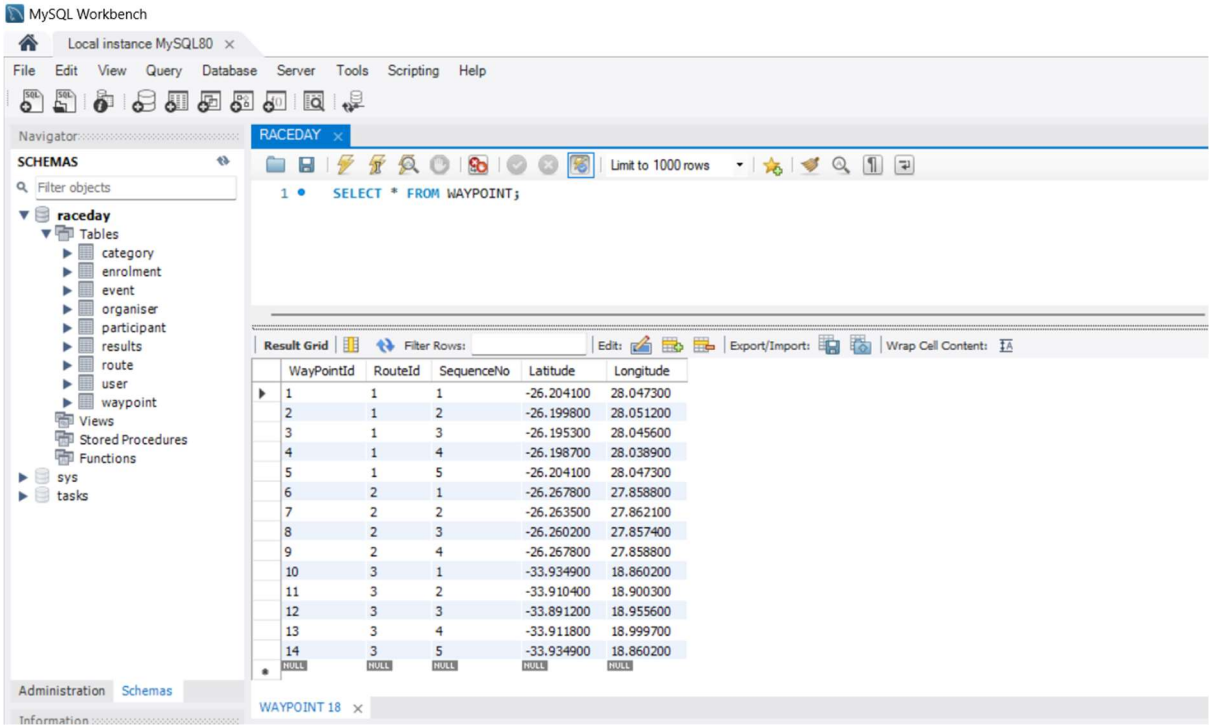
The insert statement adds one route per event with real starting coordinates and elevation gain.



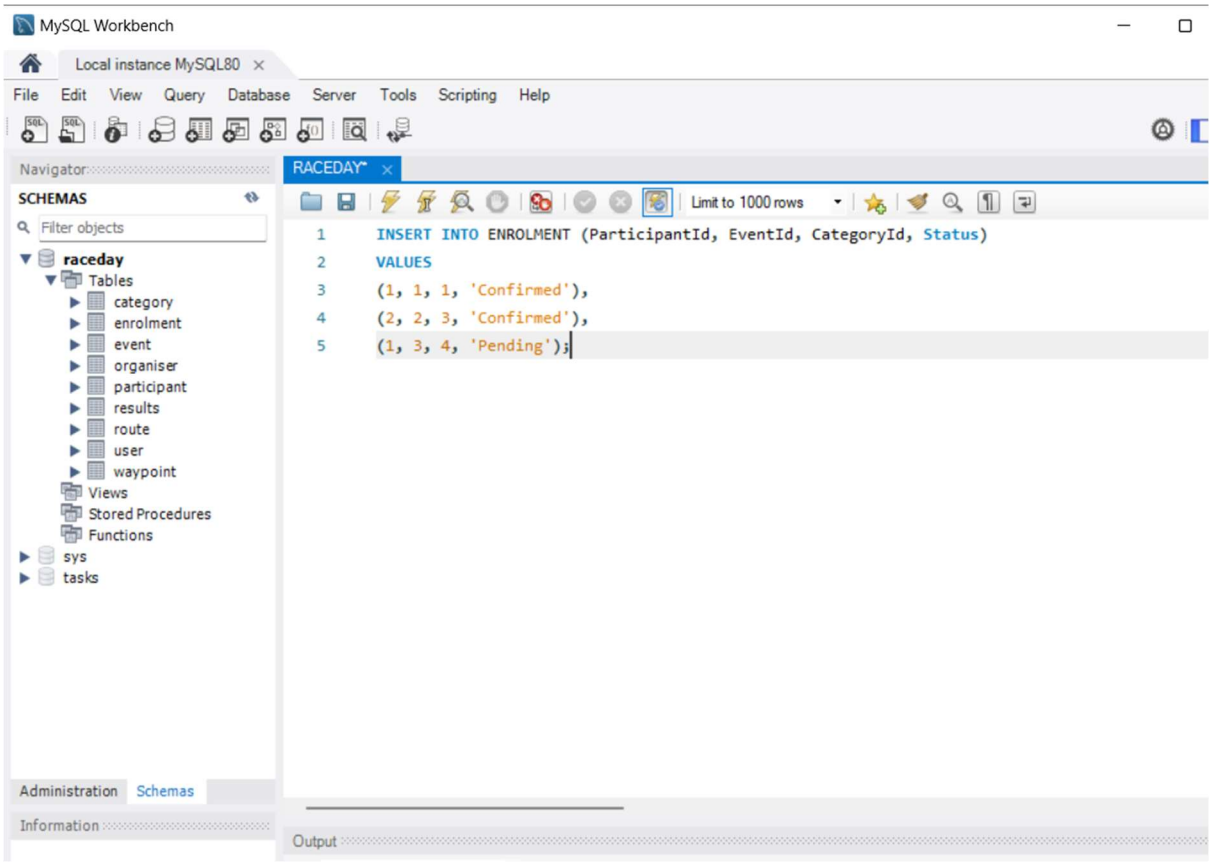
The select statement displays the set routes per event.



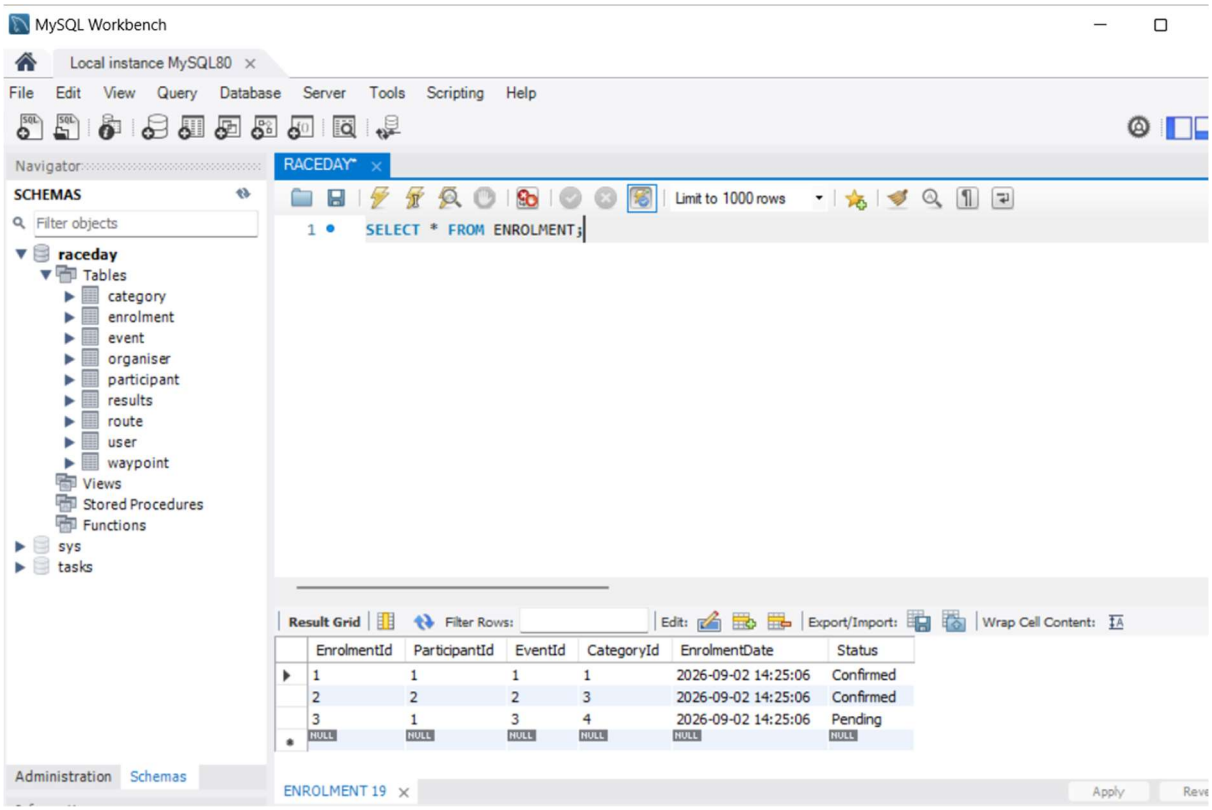
The insert statement adds an ordered set of way points per route, so each route traces a realistic closed loop when drawn on a map.



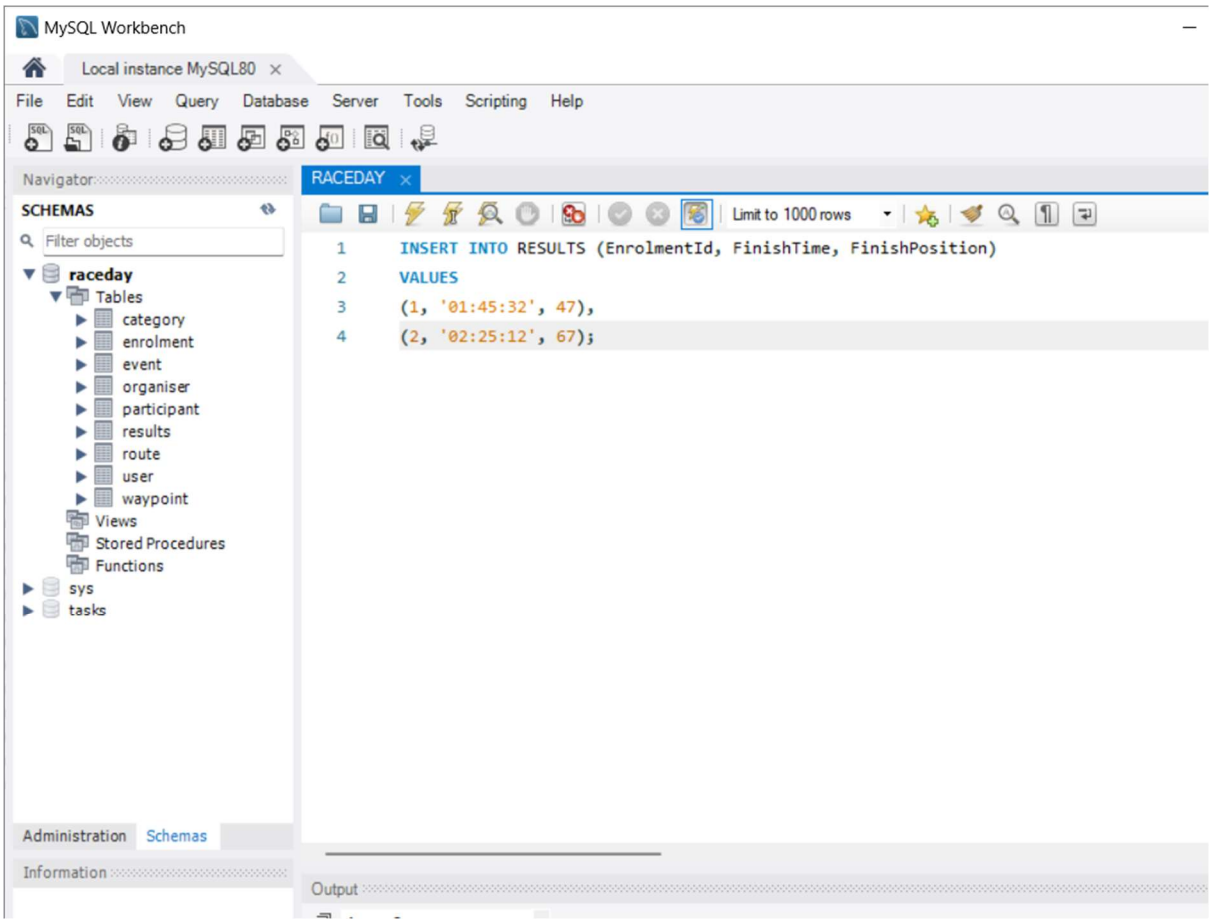
The select statement display the way points set for each route.



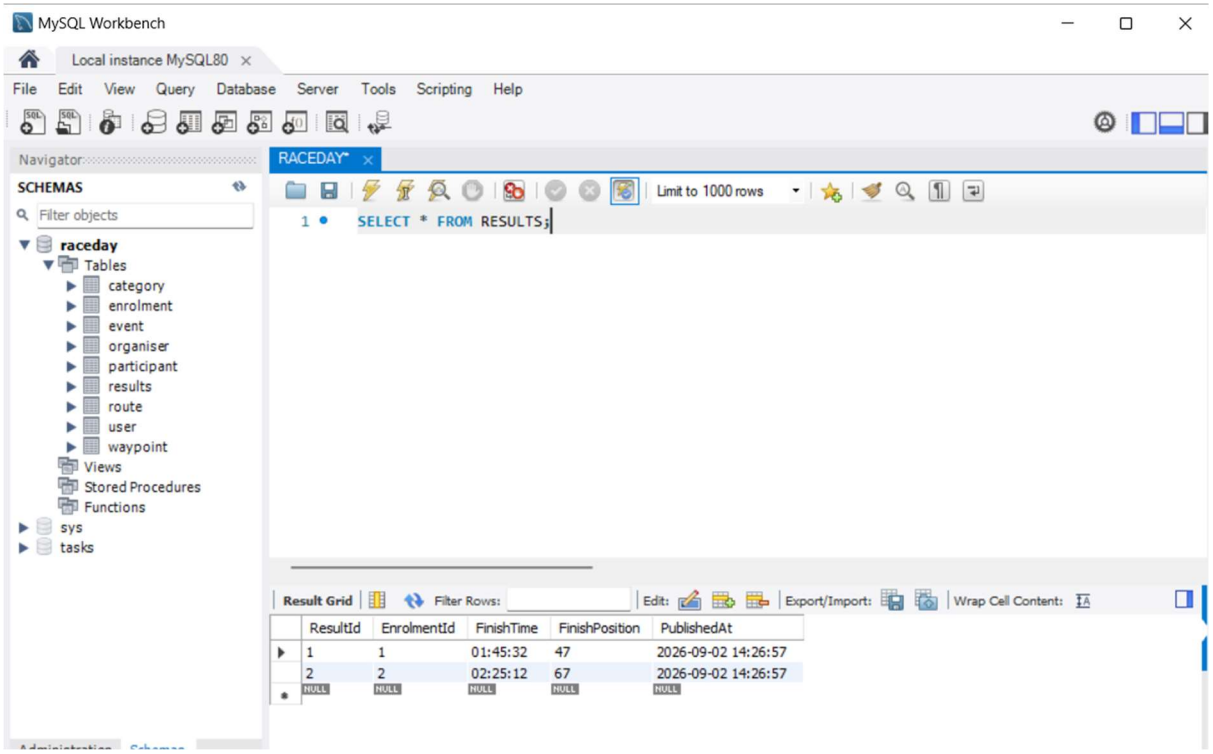
The insert statement adds a sample enrolment linking participants to events through the participant id as a foreign key and their chosen categories.



The select statement display the enrolment samples stored.



The insert statement adds samples of results for the first and second enrolment.



The select statement display the stored results samples in the table.

References

- EdPrice-MSFT (2023). *Web API design best practices - azure architecture center*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design [Accessed 28 Aug. 2026].
- Morris, C., Morris, S. and Crockett, K. (2020). *Database principles : Fundamentals of design, implementation, and management.* 3rd ed. Cengage.
- Openweathermap.org. (2026). *Current weather data*. [online] Available at: https://openweathermap.org/api/current?collection=current_forecast [Accessed 30 Aug. 2026].
- openweathermap.org. (n.d.). *One call API: Weather data for any geographical coordinate - OpenWeatherMap*. [online] Available at: https://openweathermap.org/api/one-call-api [Accessed 31 Aug. 2026].
- W3schools (n.d.). *MySQL foreign key constraint*. [online] www.w3schools.com. Available at: https://www.w3schools.com/mysql/mysql_foreignkey.asp [Accessed 1 Sept. 2026].